This cover page is to ease Projects to review document prior to submission to the Integrated Project System (IPS). A cover page, containing the document label and names of endorsing and approving parties, representing the Ordinary Electronic Signature (OES), will be automatically generated from the IPS. Project should remove this page before submission for endorsement/approval in IPS.

SOFTWARE DESIGN DESCRIPTION
FOR
[CSCI NAME]
OF
[PROJECT NAME]

Amendment Record

S/N	Description/Reason for Amendment	Page(s) Affected/ Type of Amendment*	Approved by (Name/Sign/Date)

*There are 3 types of amendment: Insertion(I), Deletion(D) and Modification(M)

TABLE OF CONTENTS

1. **SCOPE** **1**
2. **REFERENCED DOCUMENTS** **1**
3. **SOFTWARE DESIGN** **1**
3.1 CSCI OVERVIEW 1
3.2 CSCI ARCHITECTURE 1
3.3 SYSTEM STATES AND MODES 2
3.4 MODULE SPECIFICATION / CLASS DESCRIPTION 3
4. **GLOBAL DATA STRUCTURES (CSCI DATA)** **4**
5. **CSCI DATA FILES** **4**
6. **REQUIREMENTS TRACEABILITY** **4**
7. **SPECIAL NOTES** **5**

---

1. SCOPE

This Software Design Description (SDD) establishes the design for the CSCI identified as [CSCI title, (abbrev)] of the [title, (abbrev)] system. The SDD describes the allocation of the CSCI requirements specified in the SRS/IRS to the software components of the CSCI. Additionally, the SDD defines the software architecture, the interface, data and processing characteristics for each software components in the CSCI design.

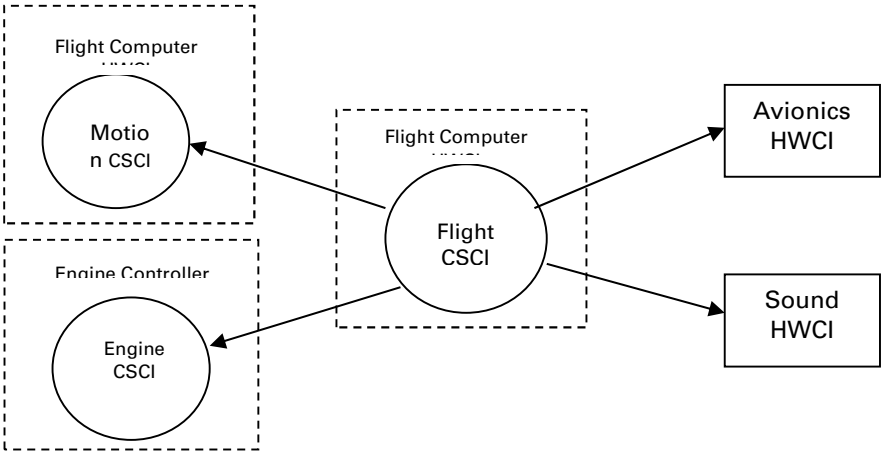
2. REFERENCED DOCUMENTS

Doc No.	Title

3. SOFTWARE DESIGN

3.1 CSCI overview

[Identity and describe the role of the CSCI within the system to which this SDD applies. A system architecture diagram may be used to show the relationships between this CSCI and the other CSCIs or HWCIs in the system.]



The purpose of the FLIGHT CSCI is to simulate the aircraft behaviour according to flight conditions, taking into account aerodynamics actions on the frame, pilot's control of the aircraft and reactions of the ground. The FLIGHT CSCI computes the position, altitude, velocities and acceleration. It verifies that the flight parameters do not exceed the aircraft limits described by the manufacturer.]

3.2 CSCI Architecture

[Describe the software architecture that form the underlying architecture for the CSCI, such as database model (e.g. relational, object oriented, hierarchical model), distributed processing (e.g. distributed/centralised database, master-slave/peer-to-peer relationship), multitasking (e.g. pre-emptive, non pre-emptive), network architecture etc.

Describe any fault/failure detection, isolation and recovery mechanisms
Describe any storage encryption or transmission encryption solution, and encryption key storage methods]

3.2.1 Structure Charts / Class Diagrams

[For Structure Design, depict the CSCI structure using a structure chart. The structure chart shows the partitioning of the CSCI into the various software components and their hierarchy, as well as the important data and control flow. Refer to SASDNotationGuideline]

[For Object-Oriented design, show all the class diagrams for this CSCI.
Refer to OOUMLAnalysisDesignNotationGuideline]

3.2.2 Text Description/ Sequence Diagrams

[For SA/SD, provide textual descriptions of the above structure chart. State the purpose of each software components, and highlight important requirements or major algorithms to be implemented.]

Example:

Module Name	Purpose
Flight	Management of aircraft stabilisation; computation of aircraft position and altitude.
Weight, Balance, & Inertia	computes the aircraft weight, balance and matrix of inertia
Flight Control	Compute forces on the aerodynamic surfaces
Aerodynamics	Computation of the aircraft aerodynamic coefficients.

[For Object Oriented design, show the sequence diagrams.
Refer to OOUMLAnalysisDesignNotationGuideline]

3.3 System States and Modes

[Identity each system state and mode in which the CSCI operates and the software components that execute in each state and mode.

A state transition table may be provided to illustrate the system states and modes that each software component executes. Make reference to SRS if the information is already documented there.

Identify the software components that executed in each state and mode.]

Example (module operation in each state/modes)

module Name	*init*	*stabili-sation*	*real-time*	*replay*	*fly out*	*snapshot*
Flight	X	X	X		X	X
Aerodynamics	X	X	X		X	X
Weight, Balance & Inertia	X	X	X		X	X
Flight Control System	X	X	X	X	X	X

Note: X means that the module operates in this mode.

[For Object Oriented design, State Transition Diagrams of one or more of the classes should be shown, if applicable

Refer to OOUMLAnalysisDesignNotationGuideline]

3.4 Module Specification / Class Description

[For SA/SD, The module specification may be in pseudo code or structured English and should contain the following information:

Name: the name of the software component should be meaningful and indicate the functions it or its subordinates serve. Indicate if the software component is time-critical or contains critical algorithms.

Input/Output data: state the input and output data of the software component.

Purpose/Design: State the purpose and design of the software component. The design should include the flow of processing and algorithms. Design constraints and alternative designs if any should also be documented.]

[For Object Oriented design, describe the class as follows:

Class Name: The name of the class should indicate the functions it serves. Indicate if this class implements a critical algorithm or is time-critical.

Class Type: This section specifies the nature of the class. It can be classified into the following types:

- Entity classes, represent those classes that have meaning in the problem domain and are likely to have long-life times
- Boundary classes, represent those things that are necessary to provide a means for actors to communicate with the system.
- Control classes, control interactions among a group of objects.

Responsibilities: Responsibilities are the attributes and operations of the class. Attributes represent stable features of a class, that is, information

about the class that must be retained to accomplish the objectives of the software specified by the customer. Attributes can often be extracted from the SOR or discerned from an understanding of the nature of the class. Operations can be extracted by performing a grammatical parse on the processing narrative for the system. Verbs become candidate operations. Each operation that is chosen for a class exhibits behaviour of the class.

[At CDR, the module specification / class descriptions should be documented, reviewed, and baselined. Subsequent changes shall be implemented using the ECP mechanism. If the module specification or class descriptions are part of source code headers (comments), they can be extracted from the source code and attached here.]

4. GLOBAL DATA STRUCTURES (CSCI DATA)

[Identify all global data used and provide the following information in the Data Dictionary. Note that data accuracy, resolution, data format etc. are only of importance if they will affect the final data which has range / accuracy / resolution requirements:

a) units of measure
b) limit/range of values (for constants provide the actual value)
c) data accuracy (e.g. ± 3 deg)
d) data resolution in terms of significant digits
e) data type (e.g. integer, real, fixed, enumerated, etc.)
f) data representation (e.g. byte, word, double word, signed or unsigned, etc.)
g) for real-time system, frequency at which the data is calculated or refreshed (e.g. 10KHz, 50 ms, etc.)
h) legality checks performed on the data (e.g. range check, format check, checksum check, etc.)
i) originator & user

These information shall be provided and baselined at CDR.]

5. CSCI DATA FILES

[Describe the database design/structure, such as files/tables definitions, and their relationship. ER diagrams and database schema diagrams should be used to depict the database design. This section shall be baselined at CDR.]

[For simple sequential data files, the description may be documented in the Data Dictionary and referenced here.]

6. REQUIREMENTS TRACEABILITY

[Trace the requirements in the SRS and/or IRS down to the software components identified in the software design, using a table or other graphical methods. This section shall be baselined at CDR.]

Example (Requirements traceability)

SDD Requirements (allocated)	Source (SRS requirements)	Description
SDD Para 3.4.1	SRS 8.0	
SDD Para 3.4.4	SRS 7.5-01	
SDD Para 3.5.2	SRS 9.5.2-01	

7. SPECIAL NOTES

[This section shall include special notes to explain any important requirements analysis, design decisions, algorithms, formula derivations, etc that have not been addressed in the preceding sections.]